Yogesh Simmhan
simmhan@iisc.ac.in

Department of Computational and Data Sciences
Indian Institute of Science, Bangalore

DS256 (3:1)

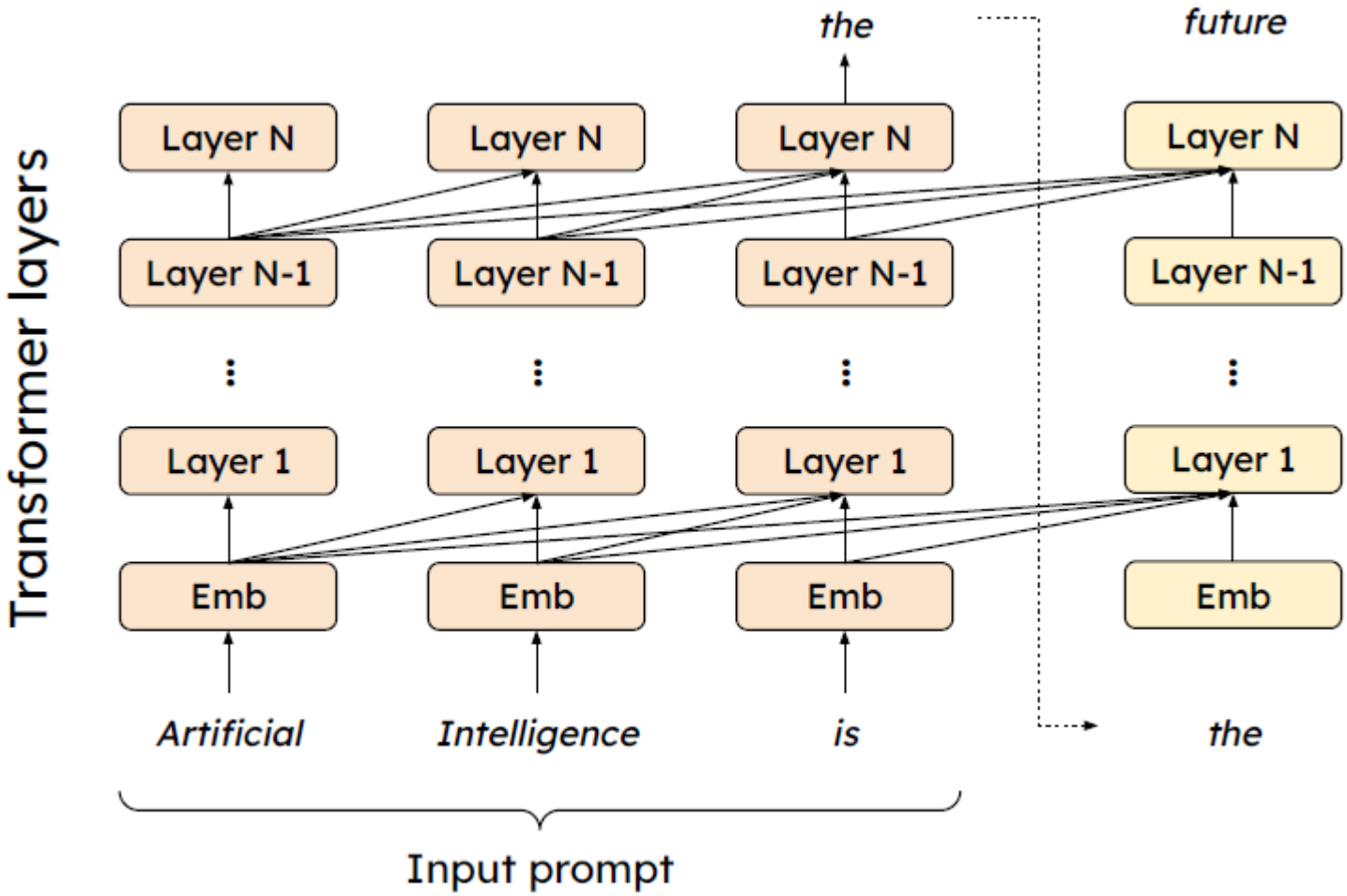
Scalable Systems for Data Science

Module 3

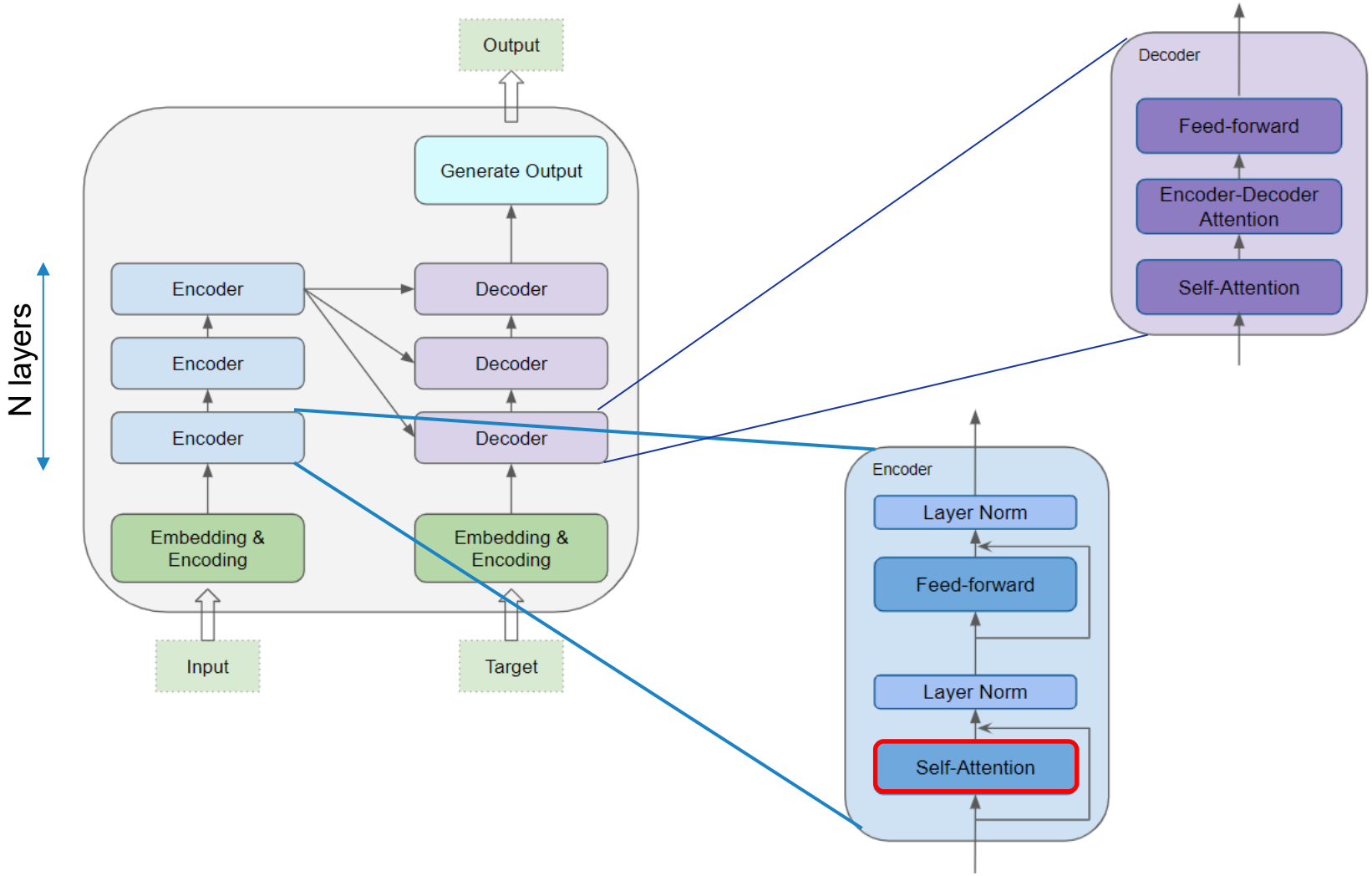
Machine Learning at Scale

A Brief Primer on Transformer Models & Attention

Inferencing using Transformer Architecture: A Rapid Review



Transformer Architecture



https://towardsdatascience.com/transformers-in-action-attention-is-all-you-need-ac10338a023a/
https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452/

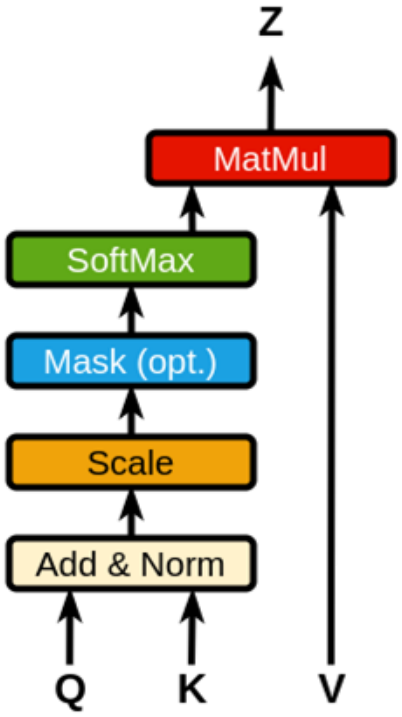
Attention & Multi-headed Attention

- **Attention** relates words in the input to each other. Model helps pay "attention" to specific prior words.
- Attention weights help infer that ***Ball*** is strongly related to ***blue*** and ***holding*** but weakly to ***boy***.
- ***it*** is closely related to ***cat*** in the first phrase but to ***milk*** in the second phrase.
- **Multiple attention heads** (weights) can help related words based on different perspectives, e.g., focus on action, description, objects, …
  - Richer understanding of context

https://towardsdatascience.com/transformers-in-action-attention-is-all-you-need-ac10338a023a/
https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452/

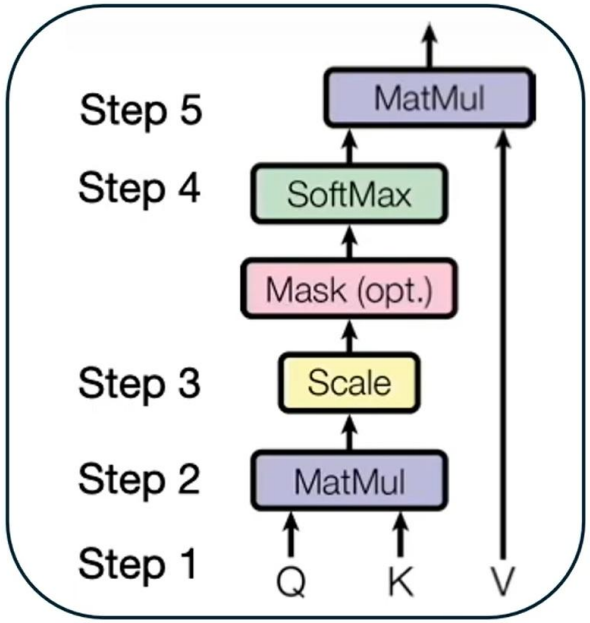
Transformer Architecture: Self Attention

- **Query (Q)**: The current token that is searching for context, e.g., query phrase to find relevant information.
- **Key (K)**: All the tokens being searched over. Model decides how much "attention" to pay to each token, how well it matches the Query.
- **Value (V)**: Actual content contained in each token. Takes values from important tokens based on attention to return final output.

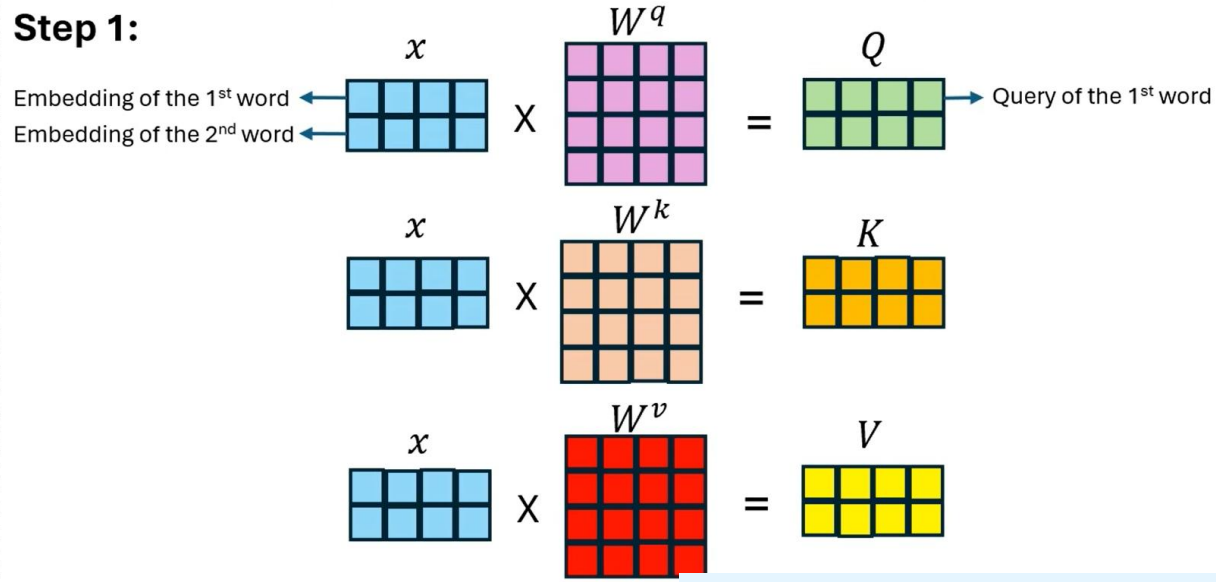


Transformer Architecture: Self Attention

Convert word to embedding, projecting it to the search/**query** space, the **key**/label space, and the **value**/content space. Weights are from the trained model.

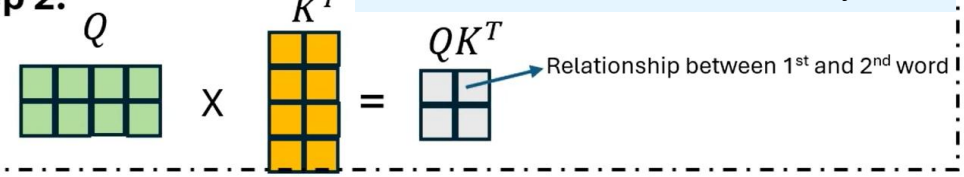


Step 1:



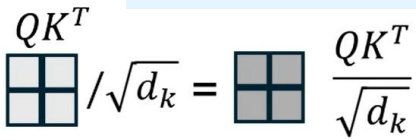
How well **Q** matches with the keys **K**?

Step 2:

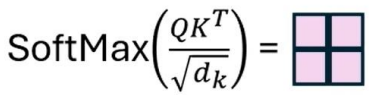


Scale down large values from dot prod of large vectors for gradient stability.

Step 3:

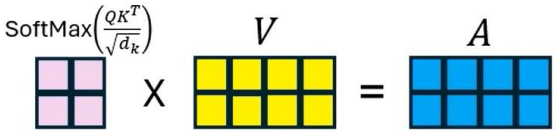


Step 4:



Convert scaled match scores into weights.

Step 5:



Size of A: $T \times E$

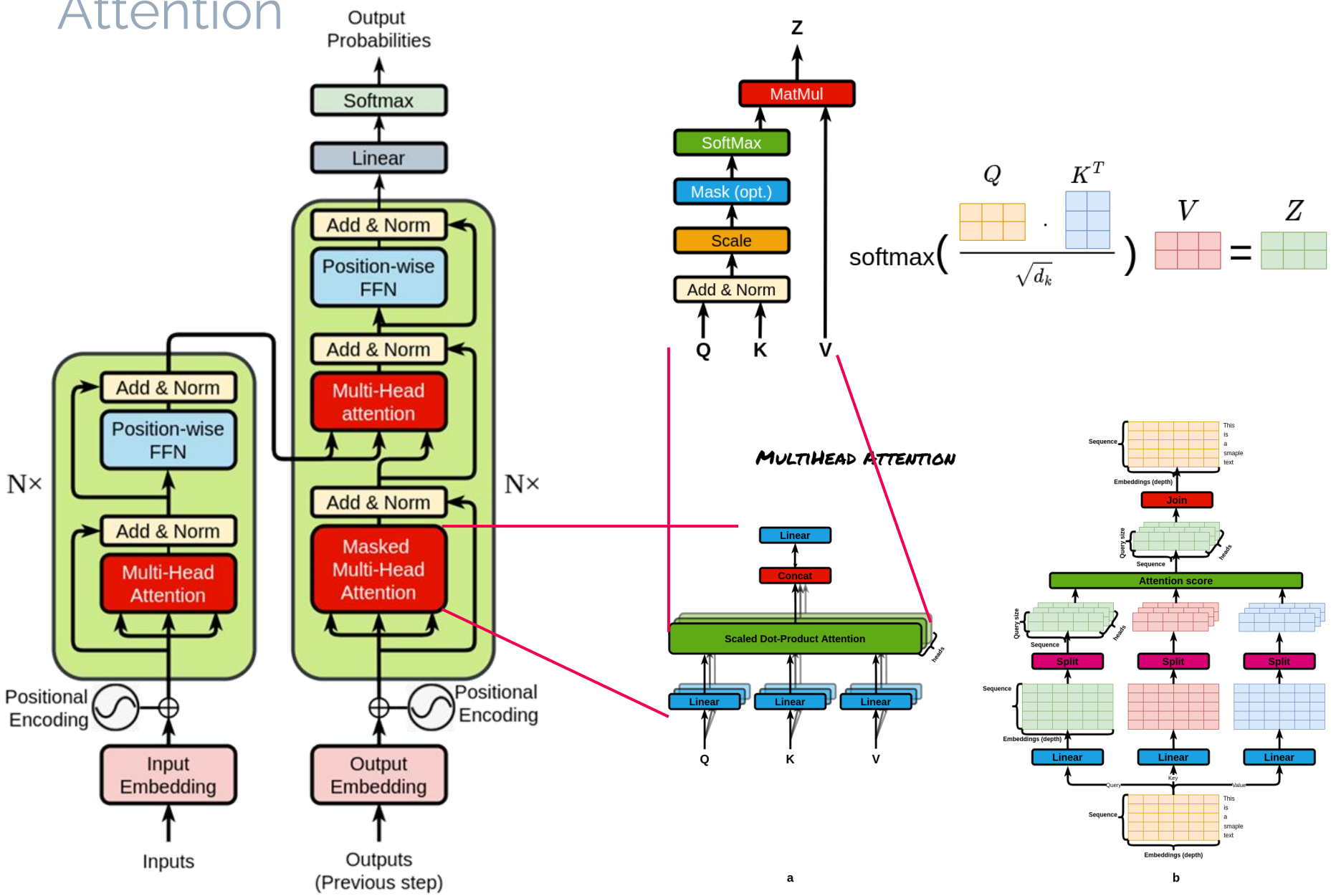
T: Number of Input Tokens

E: The Embedding Length

Weighted average of all Values, Selects best (future) token to match the tokens in the query

https://www.youtube.com/watch?v=W3n07cxydeQ&t=226s

Transformer Architecture: Self and Multi-headed Attention



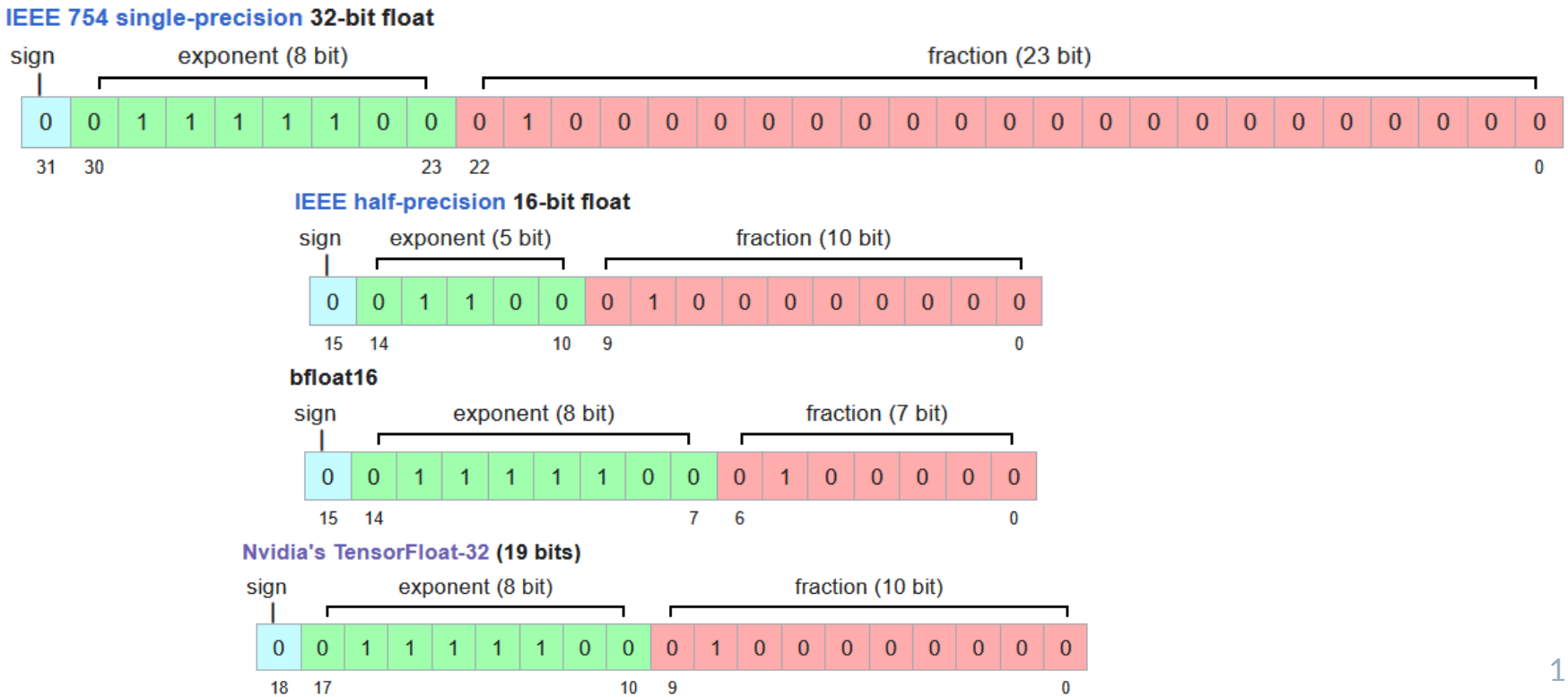
https://towardsdatascience.com/transformers-in-action-attention-is-all-you-need-ac10338a023a/
https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452/

The Computational Load

- **Arithmetic Intensity (AI)**: Ratio of floating-point operations (**FLOPs**) performed to bytes **transferred** from memory
- **Roofline model**
  - High AI ➲ Compute Bound
  - Low AI ➲ Memory (bandwidth) Bound
- GPUs skew more towards compute speed than memory bandwidth
  - NVIDIA H100 has ~1,000 TFLOPS of BF16 compute but only 3.35 TB/s of memory bandwidth, ~330 FLOPs/byte
  - Roofline model: AI > 330 ➲ App is compute bound on H100, else memory bound

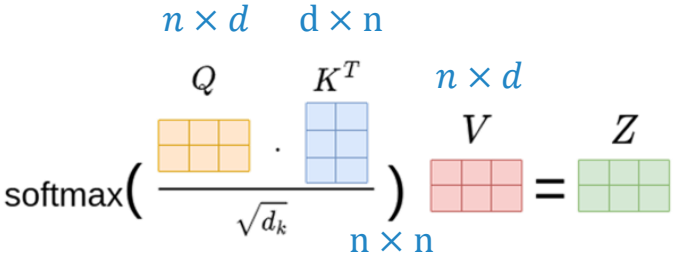
Side Note: Number Format Precision

- BF16 (Brain Floating Point), 2 bytes space (Google)
  - Shorter than IEEE FP32, can be used with mixed precision
  - Retains 8bit exponent, reduced 8bit precision
  - Similar to TF32 (19bits) from Nvidia for Tensor Core

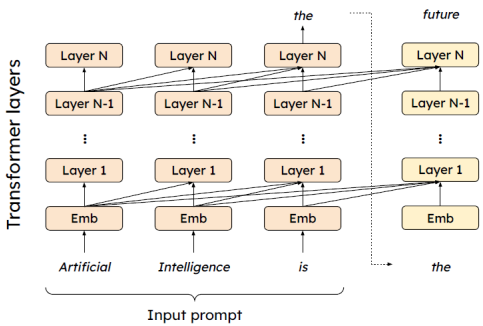


https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

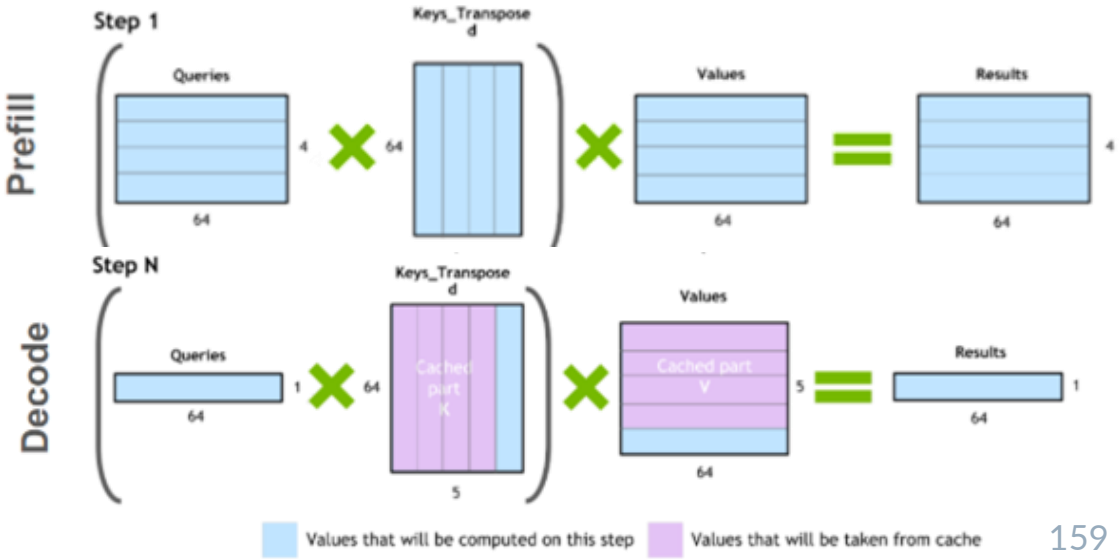
Prefill & Decode Phases



- **Prefill Stage** has a larger set of input tokens. Can perform batch operations on SM(Q*K')*V…more *compute intensive*
  - $O(n^2.d)$, with *n* input tokens, *d* embedding size
- **Decode Stage** is performed one new token at a time…less compute ($O(t)$ for t^{th} token, with caching) but more *memory intensive* due to inc. mat. size.



Prefill ***Decode >>***



https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/

Prefill & Decode Phases

- Even with multiple requests batched together, decode batch size unable to saturate GPU compute…
  - 1000s of requests need batching across prompts, typically not possible
  - GPU spends time waiting for weight transfers rather than performing computation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Key-Value Cache

- Computing attention for all previous tokens for every new token is computationally costly, redundant
- **KV Cache** stores and reuses prior key and value
- Avoids recomputation…
- But adds to **memory usage** on GPU, needs bytes:

$$\text{KV cache per token} = 2 \times L \times H \times d \times \text{sizeof(dtype)}$$

- *L* layers, *H* attention heads, *d* dim of head
- *E.g., 100 layers, 8 key-value heads, 128 head dimension. Each token needs ~400 KB. Request with 100,000 tokens uses* ***40GB*** *KV cache.*

Key-Value Cache

- **Memory Bandwidth** also increases when accessing the KV cache for each token
- **Memory fragmentation** due to growing nature of KV cache with number of tokens & dynamic allocation
- Larger Batches ➲ Better AI and GPU usage ➲ Larger KV Cache memory usage

Performance Metrics

- **Time-to-first-token (TTFT):** Latency from request submitted till first output token is generated
  - Dominated by the prefill phase…*compute bound*
  - Affected by prompt length (higher compute), queuing delays due to prior prompts, prefill batching with earlier decode phase
- **Time-per-Output-Token (TPOT)**: Time to next token in decode phase. Speed of generation…
  - Decided by decode latency…*memory bound*
  - Affected by batch size (higher compute), sequence length seen so far (more KV cache access)
- TTFT of ~2 secs, TPOT of 200ms for interactive

Orca: A Distributed Serving System for Transformer-Based Generative Models

OSDI 2022

LLM Service Systems

- **Key challenge:** Serving systems like Nvidia Triton that use *request-level scheduling* cannot adapt to *variable-length decoding* workloads
  - Scheduler creates a **batch of requests** from a queue…improves AI, makes both prefill and decode more compute bound
  - Schedules batch to execution engine
  - Engine processes batch over **multiple iterations** of the model
  - Returns the generated text back to the users

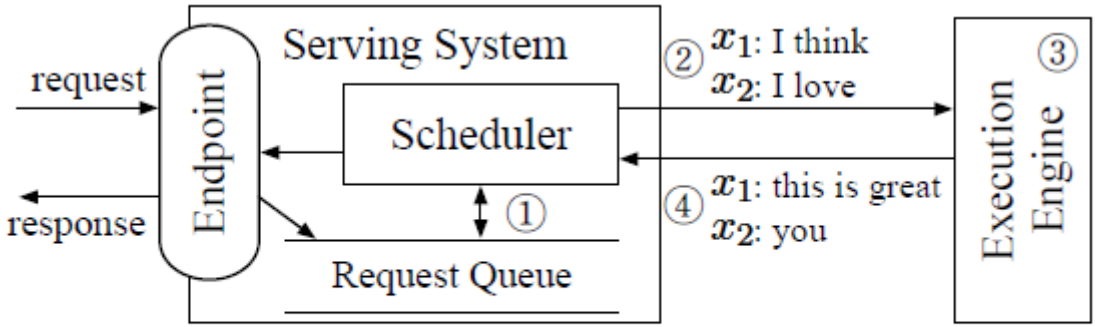


Figure 2: Overall workflow of serving a generative language model with existing systems.

Challenge: Early Finish, Late Join

- Keep batches fixed until all requests complete
- Performs (unnecessary) operations on all requests, even after some reach EOS
- Early-finished requests wait unnecessarily for rest to complete
- New request waits for current batch to complete

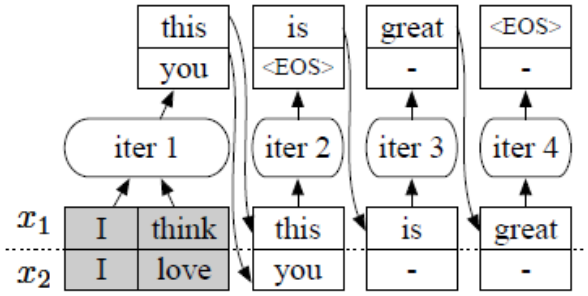


Figure 3: An illustration for a case where the requests have the same input length but some requests finish earlier than others. Shaded tokens represent input tokens. "-" denotes inputs and outputs of extra computation imposed by the scheduling.

Solution: Iteration-level scheduling in Orca

- Schedules at the iteration granularity
  - Runs only one iteration per scheduling action

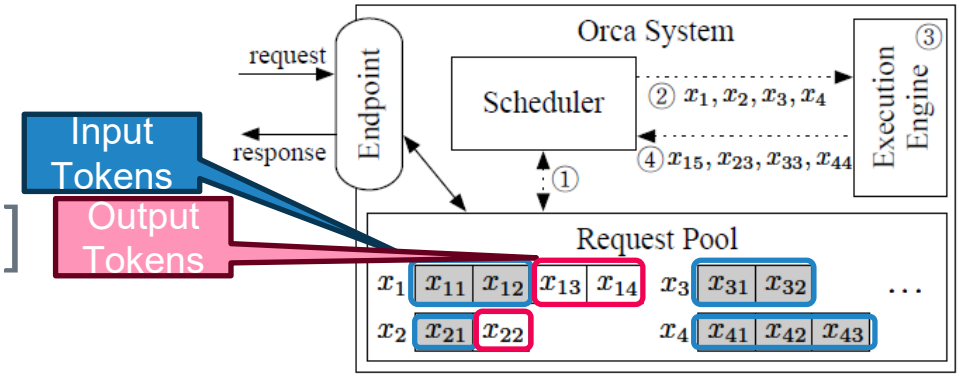
1. Selects requests to run next
2. Invokes the engine to execute one iteration for the selected requests
3. Receives execution results for the scheduled iteration

- **Immediate Return:** Detects completion of a request and returns generated tokens to client
- **Fast Admission:** Newly arrived request gets chance to start processing in next iteration

Challenge: Batching an arbitrary set of requests

- Batching requests is necessary for high efficiency
- To batch the execution of multiple requests, requests must have identical operations with same tensor shape
- But requests at *different token indices* produce tensors with incompatible shapes, preventing batched execution

Challenge: Batching an arbitrary set of requests

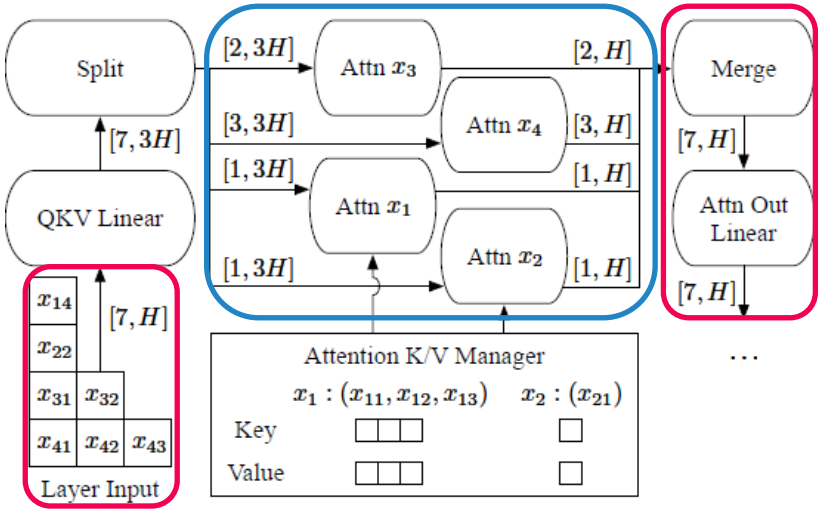


- ▷ Transformer layer [B,L,H]
- ▷ For a pair of requests

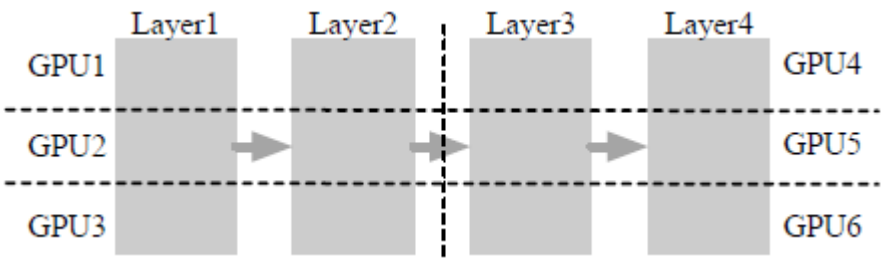
1. Both are in the initiation phase and each has *different number of input tokens* ***L*** (e.g., x3 and x4)
   - *Input tensor lengths different*
2. Both are in the increment phase and each is processing token at *different index* (x1 and x2)
   - *Different shape of Attention KV tensor*
3. Each request is in the different phase: initiation or increment (x1 and x3).
   - *Init phase takes all input tokens, Incr only one more*

Solution: Selective batching in Orca

- Don't batch all available requests
- Non-attention layers can be flattened into token-wise batching [7,H]
  - Benefits from model param reuse
- Split into separate processing of Attention layers
  - No batching for Attention
  - Attention does not benefit from model param reuse
  - No locality in KV cache

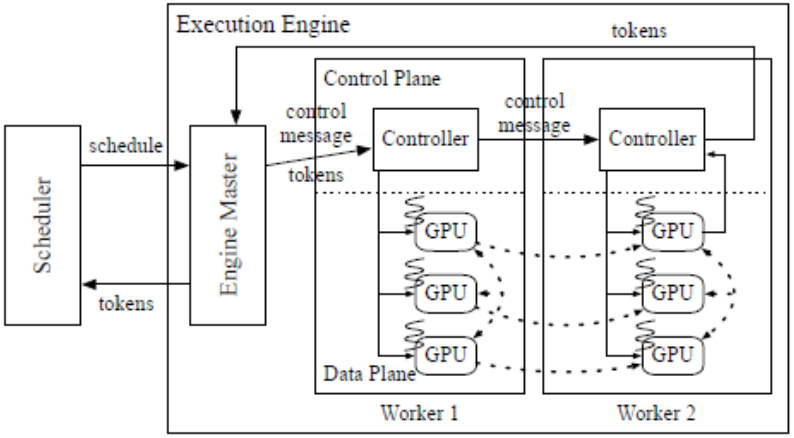


Orca Design



- Intra-layer parallelism
  - Split linear and attention layers across GPUs
- Inter-layer parallelism
  - Splits transformer layers across GPUs
  - Load balances layers across each GPU
- Worker performs ops on inter-layer model partition
- Dedicated CPU thread of each part

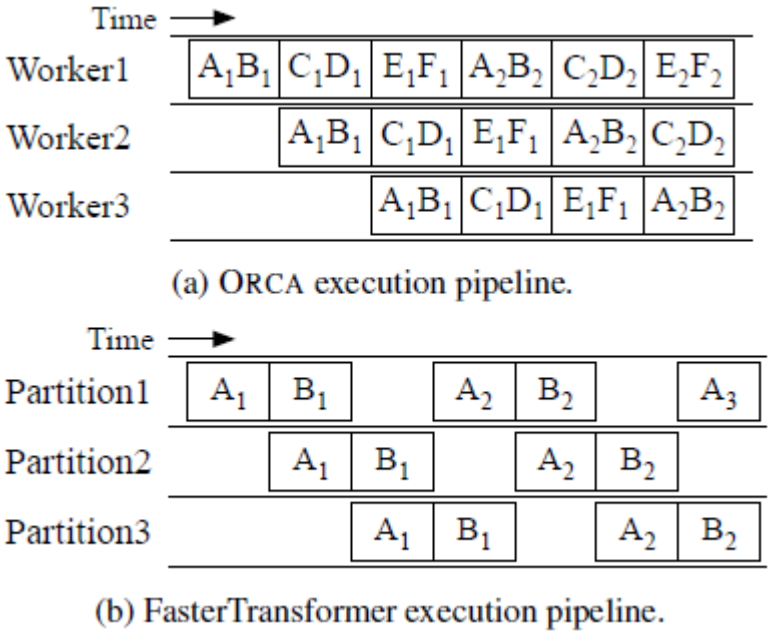
Orca Design



- *Engine master* forwards received info about scheduled batch to the first worker process
  - *Tokens, request IDs, index, Token count*
- Controller passes to GPU-controlling threads, issues GPU kernels calls to its GPU
  - E.g., Access appropriate GPU memory for KV cache
- Controller async initiates Worker 2
  - W2 waits for outputs from W1
  - Sends response back to Engine Master after exec

Scheduling & Pipeline Parallelism

- Iteration-level FCFS
  - If a request arrives earlier than other, it will have processed at least as many iterations as the later one



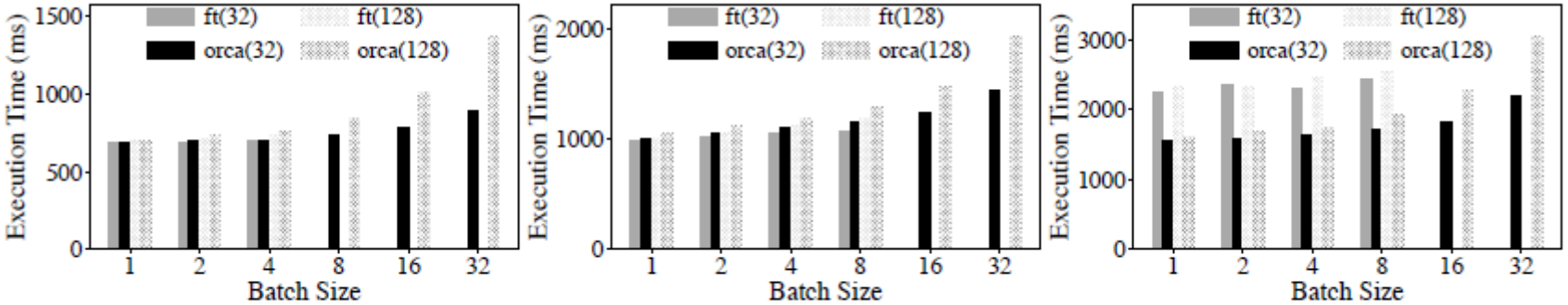
(a) ORCA execution pipeline.

(b) FasterTransformer execution pipeline.

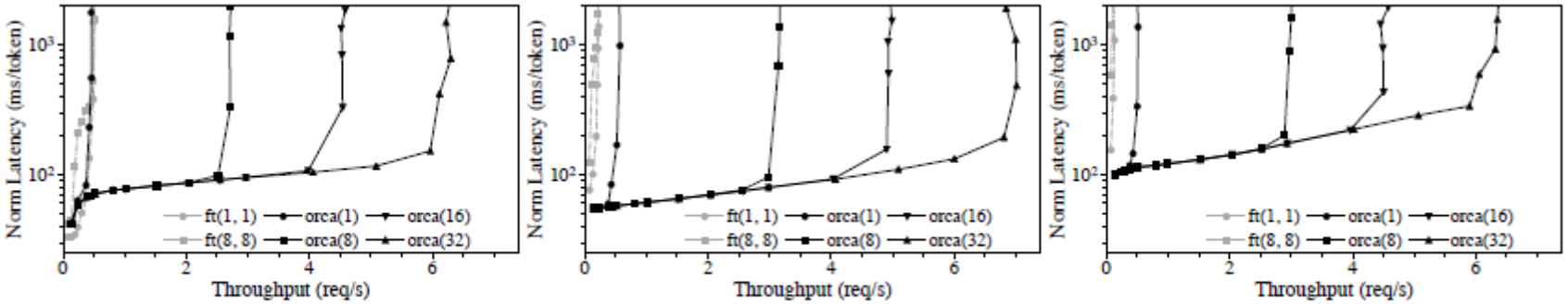
- Max-batch to tradeoff between throughput vs. latency
- GPU memory reservation for init phase KV slots. Avoid request batch overflow.
- Execution of workers in engine pipelined across batches…keeps concurrently running batches to be n_workers

Results

- Without and With Scheduler



(a) 13B model, 1 GPU. (b) 101B model, 8 GPUs. (c) 175B model, 16 GPUs.



(a) 101B model, 8 GPU. (b) 175B model, 16 GPUs. (c) 341B model, 32 GPUs.

▷ *End of Module*